Having explored how language models assign probabilities to sequences of text, we now arrive at one of the foundational pillars of applied NLP: Text Classification. Whether we want to detect spam, analyze customer sentiment, categorize news articles, or identify grammatical roles, the fundamental objective remains the samemapping an input text x to a discrete label or category y^*.

When formulating this task mathematically, machine learning offers two fundamentally distinct philosophies for predicting the optimal class y^*: Generative (Joint) Modeling and Discriminative (Conditional) Modeling.

Classification: Two Foundational Approaches

1. Joint / Generative Models (e.g., Naïve Bayes)

The generative philosophy seeks to understand how the data itself was created. Instead of jumping directly to the label, the model learns a full probabilistic story of the world by capturing the joint probability distribution $P(x, y)$.

- Core Mechanism:
  - Typically assumes a specific functional form for the prior probability of the class $P(y)$ and the likelihood of observing the features given that class $P(x \mid y)$.
  - Estimates these parameter distributions directly from the training data (typically via counting and MLE).
  - Uses Bayes' Rule at test time to invert the conditioning and compute the posterior probability $P(y \mid x)$.
- Inference:

$$\begin{aligned} y^* &= \arg\max_y P(y \mid x) \\ &= \arg\max_y \frac{P(y)P(x \mid y)}{P(x)} \\ &= \arg\max_y P(y)P(x \mid y) \end{aligned}$$

- (Notice that the marginal probability of the evidence $P(x)$ is constant with respect to y, so it drops out of the arg max computation.)
- Key Advantages:
  - Learning is generally easy and fast: Estimating parameters often boils down to simple empirical counting rather than iterative numerical optimization.
  - Smoothing is well-understood: Classical smoothing techniques (like Laplace, absolute discounting, or interpolation) transfer cleanly to prevent zero-probability errors on rare features.
  - Complete model of the data: Because it models $P(x, y)$, it can also be used to sample synthetic examples x or handle missing attributes naturally.

2. Conditional / Discriminative Models (e.g., Logistic Regression)

The discriminative philosophy takes a pragmatic shortcut: if our ultimate goal is merely to decide which label y belongs to an observed text x, why waste model capacity learning how to generate the text x in the first place? Instead, it models the decision boundary directly via the conditional distribution $P(y \mid x)$.

- Core Mechanism:
  - Directly parameterizes and models $P(y \mid x)$ without concerning itself with the generative distribution of inputs.
  - Optimizes the parameters explicitly to maximize the conditional likelihood of the correct labels given the observed text.
- Inference:

$$y^* = \arg\max_y P(y \mid x)$$

- (Prediction is performed directly without needing Bayes' rule or prior normalization.)
- Key Advantages:
  - No need to model $P(x)$: Sidesteps the complex and often unrealistic independence assumptions required to model high-dimensional feature distributions $P(x \mid y)$.
  - Flexible, feature-rich representations: Allows engineers to freely design overlapping, highly correlated, and arbitrary features (e.g., n-grams, capitalization patterns, word shape, syntactic parse signals) without worrying about violating generative independence assumptions.

Now that we have established the conceptual split between generative and discriminative views, the natural next step is parameter estimation: given training data, how do we actually learn the parameters θ?

In statistical machine learning, the standard principle is Maximum Likelihood Estimation (MLE).

Likelihood and Conditional Likelihood

The likelihood measures the probability of observing our training dataset under a chosen set of parameters θ:

- Notation:
  - $x = \{x_1, \ldots, x_N\}$: The set of observed input samples (features).
  - $y = \{y_1, \ldots, y_N\}$: Their corresponding target labels.
  - θ: The model parameters.
- Generative Likelihood (Joint): In a joint or generative model, we assess the probability of observing both inputs and labels:

$$L(\theta) = P(x, y \mid \theta) = \prod_{i=1}^{N} P(x_i, y_i \mid \theta)$$

- Discriminative Likelihood (Conditional): In a discriminative model, we do not model how x was generated; instead, we evaluate the probability of the true labels conditioned on the observed inputs:

$$L_{\text{cond}}(\theta) = P(y \mid x, \theta) = \prod_{i=1}^{N} P(y_i \mid x_i, \theta)$$

Maximum Likelihood Estimation (MLE)

The objective of MLE is intuitive: find the specific parameter values θ^* that make our observed training data as likely as possible.

- General / Joint MLE Formulation:

$$\theta^*_{\text{MLE}} = \arg\max_{\theta} L(\theta) = \arg\max_{\theta} P(x, y \mid \theta) = \arg\max_{\theta} \sum_{i=1}^{N} \log P(x_i, y_i \mid \theta)$$

- Discriminative (Conditional) MLE Formulation:

$$\theta^*_{\text{CMLE}} = \arg\max_{\theta} L_{\text{cond}}(\theta) = \arg\max_{\theta} P(y \mid x, \theta) = \arg\max_{\theta} \sum_{i=1}^{N} \log P(y_i \mid x_i, \theta)$$

Key Theoretical Properties of MLE

1. Broadly Applicable: MLE serves as a foundational estimation approach across statistical modeling, classical machine learning, and deep learning.
2. Statistical Consistency: Under mild regularity conditions, MLE is consistentas the number of independent and identically distributed (i.i.d.) training samples tends to infinity ($N \to \infty$), the estimator converges in probability to the true underlying parameter value θ^*.
3. The Overfitting Risk: In small sample regimes (data scarcity), pure MLE places extreme confidence in sparse observations, leading directly to high variance and overfitting (e.g., zero-count probabilities). Regularization or Bayesian priors (MAP estimation) are often required to stabilize it.

With the general formulation of generative modeling established, we arrive at its classic baseline: the Naïve Bayes classifier. Despite its radical simplifying assumptions, it remains a fast, effective, and surprisingly robust benchmark for text categorization tasks like spam filtering and sentiment analysis.

The Naïve Bayes Model

- Feature Representation: Represent each input instance x as a vector of d discrete features

$$x = (x^{(1)}, x^{(2)}, \ldots, x^{(d)}) \in \text{VALUES}^d$$

  - (where VALUES is a finite set of possible feature values, such as binary presence $\{0, 1\}$ or word counts).

- Discrete Label Space: The target label belongs to a finite set of classes:

$$y \in L$$

The "Naïve" Conditional Independence Assumption

Under the chain rule of probability, the joint distribution factors exactly as:

$$P(x, y) = P(y)P(x \mid y) = P(y)P(x^{(1)}, \ldots, x^{(d)} \mid y)$$

Modeling full arbitrary correlations between all d features given the class y is computationally intractable and severely data-starved (requiring $|\text{VALUES}|^d$ parameters per class).

The core assumption of Naïve Bayes resolves this by assuming that all features are conditionally independent given the class label y:

$$P(x^{(1)}, \ldots, x^{(d)} \mid y) = \prod_{j=1}^{d} P(x^{(j)} \mid y)$$

Thus, the full generative model is defined as:

$$P(x, y) = P(y) \prod_{j=1}^{d} P(x^{(j)} \mid y)$$

Prediction & Inference

Given a new test instance x, we apply Bayes' rule to identify the most probable class y^*:

$$y^* = \arg\max_{y \in L} P(y \mid x) = \arg\max_{y \in L} \frac{P(x, y)}{P(x)} = \arg\max_{y \in L} P(y) \prod_{j=1}^{d} P(x^{(j)} \mid y)$$

- Computational Simplicity: Because the label space L is a small finite set (e.g., {spam, ham} or {positive, negative, neutra inference requires no complex searchwe simply iterate exhaustively over each possible label $y \in L$, evaluate the product score, and select the maximum.
- Numerical Implementation Note: In practice, multiplying hundreds or thousands of probabilities leads to floating-point underflow. Inference is almost universally executed in log-space:

$$y^* = \arg\max_{y \in L} \left[\log P(y) + \sum_{j=1}^{d} \log P(x^{(j)} \mid y) \right]$$

Now we apply the Naïve Bayes framework directly to Text Classification.

Text Classification & Feature Representation

- Primary Objective: Assign an input document to one (or more) predefined categories (e.g., topic labeling, spam detection, sentiment).
- The Classical Representation (Bag of Words): Traditionally, documents are represented using a Bag of Words (BoW) modeltreating the document simply as a collection of word counts or indicators, deliberately ignoring grammar and word order.
- Additional Information Sources: While word identity is the primary signal, other features can be incorporated:
  - Document length
  - Average word length
  - Metadata (source, author, timestamp)
  - Document layout and visual/formatting cues

Bag of Words (BoW) Feature Encoding: An Example

Suppose we use binary indicators ($x^{(j)} \in \{0, 1\}$) denoting whether a vocabulary word appears in the document:

- Vocabulary (Feature Space):

$$\text{Features} = [\text{yossi, dina, dana, likes, apples, oranges, and, both}]$$

- Sample Vectorization:

Sentence	Feature Vector Representation
"yossi likes apples"	[1, 0, 0, 1, 1, 0, 0, 0]
"dina likes oranges"	[0, 1, 0, 1, 0, 1, 0, 0]
"dana likes both apples and oranges"	[0, 0, 1, 1, 1, 1, 1, 1]

This representation works reasonably well for coarse-grained topical categorization, where word presence strongly correlates with the subject matter.

Prediction in Multivariate Bernoulli Naïve Bayes

Given a trained model and a document x, we predict the most likely category y^* using the MAP decision rule:

$$y^* = \arg\max_{y \in L} P(y)P(x \mid y) = \arg\max_{y \in L} P(y) \prod_{j=1}^{d} P(x^{(j)} \mid y)$$

When features are binary word indicators, we can split the product into two distinct sets: the words that appear in the document ($x^{(j)} = 1$) and the words that do not appear ($x^{(j)} = 0$):

$$y^* = \arg\max_{y \in L} P(y) \left(\prod_{j:\text{word}_j \in x} P(x^{(\text{word}_j)} = 1 \mid y) \right) \left(\prod_{j:\text{word}_j \notin x} P(x^{(\text{word}_j)} = 0 \mid y) \right)$$

- Crucial Detail: In this multivariate Bernoulli formulation, the non-occurrence of a word carries explicit probabilistic evidence just like its presence does (since $P(x^{(j)} = 0 \mid y) = 1 - P(x^{(j)} = 1 \mid y)$).

Learning in Bag of Words Naïve Bayes

Given N labeled training examples $\{(x_1, y_1), \ldots, (x_N, y_N)\}$:

Model Parameters

1. Class Prior Probabilities: $P(y)$ for each category $y \in L$.
2. Feature Likelihoods: The probability of each feature given each class:

$$\theta_{j|y} = P(x^{(j)} = 1 \mid y)$$

Maximum Likelihood Estimators (MLE)

Maximizing the joint log-likelihood over the N training examples leads to intuitive frequency ratios:

- Prior Estimator:

$$\hat{P}(y) = \frac{\text{Count}(y)}{N}$$

- Feature Likelihood Estimator:

$$\hat{P}(x^{(j)} = 1 \mid y) = \frac{\text{Count}(y \text{ and } x^{(j)} = 1)}{\text{Count}(y)}$$

- (The fraction of documents in class y that contain feature j).
- The Need for Smoothing: If a word in the vocabulary never appears in any class-y training document, its MLE probability is exactly 0:

$$\hat{P}(x^{(j)} = 1 \mid y) = 0$$

If that word later appears in a test document, the entire class product becomes 0. Therefore, smoothing (such as Laplace/Add-1 smoothing) is essential:

$$\hat{P}_{\text{smoothed}}(x^{(j)} = 1 \mid y) = \frac{\text{Count}(y \text{ and } x^{(j)} = 1) + 1}{\text{Count}(y) + 2}$$

The Discriminative Approach

Limitations of Generative Models with Complex Features

Generative models (such as Naïve Bayes) become difficult to specify and train when inputs are characterized by complex, overlapping, or interdependent features:

- The Independence Dilemma: A generative model requires estimating the feature likelihood $P(x \mid y)$. To make this computationally feasible, Naïve Bayes enforces the conditional independence assumption $P(x \mid y) = \prod_j P(x^{(j)} \mid y)$.
- Correlated Features in Text: Real-world text features are heavily correlated:
  - Unigram and n-gram Co-occurrence: Using both the individual word "York" (unigram) and the phrase "New York" (n-gram) as simultaneous features produces double-counting of evidence, falsely inflating confidence.
  - Morphological & Syntactic Overlaps: Combining word identities, capitalization flags (e.g., is_capitalized), suffixes (-ing), and part-of-speech tags introduces severe structural dependencies.
- Failure Mode: When highly correlated features are added to a generative model, the independent likelihood terms multiply redundant evidence, leading to extreme, uncalibrated probability estimates.

The Discriminative Shift

Discriminative models bypass the need to model the distribution over inputs $P(x)$ entirely. By directly targeting the conditional distribution:

$$P(y \mid x)$$

they treat the input x as fixed, observed conditioning context.

- Arbitrary Feature Engineering: Because x is given, no probabilistic independence assumptions among the features in x are required. Engineers can define arbitrary, non-independent, and overlapping feature functions without distorting the underlying model.
- Log-Linear Models: While many discriminative classification techniques exist (e.g., Support Vector Machines, Decision Trees, Random Forests), log-linear models (also known as Multinomial Logistic Regression or Maximum Entropy models) remain a dominant classical baseline in the NLP community and serve as the direct mathematical precursor to neural network output layers (Softmax).

Feature Representation in Discriminative Models

In discriminative modeling, instead of defining a single feature vector for the input x alone, we define features over pairs of inputs and labels (x, y):

- Indicator Functions: Features are typically indicator functions that count or flag the co-occurrence of specific patterns in the input x alongside a candidate class label y.
- Label-Dependent Values: Because the features evaluate the pair (x, y), different candidate classes produce distinct feature activations for the exact same input string.

The Block Notation

A standard way to conceptualize the joint feature mapping $\phi(x, y)$ across a discrete label set $L = \{y_1, y_2, \ldots, y_{|L|}\}$ is using block vectors:

1. Extract a base feature representation of input x (e.g., presence of words in a vocabulary).
2. The joint feature vector $\phi(x, y)$ has length $|L| \times d$ (where d is the number of base features).
3. The base vector is placed into the specific "block" corresponding to label y, while all other label blocks are filled with zeros.

Worked Example

Consider classifying the sentence fragment:

... win the election ...

Suppose our base vocabulary features are the indicator flags for 4 vocabulary words:

["win", "ball", "election", "game"]

For this text, the base activation vector is [1, 0, 1, 0]. If our candidate classes are $L = \{\text{SPORTS, POLITICS, OTHER}\}$, the resulting block feature vectors are:

$$\begin{aligned}\phi(x, \text{SPORTS}) &= [\mathbf{1}, \mathbf{0}, \mathbf{1}, \mathbf{0}, \quad 0, 0, 0, 0, \quad 0, 0, 0, 0] \\ \phi(x, \text{POLITICS}) &= [0, 0, 0, 0, \quad \mathbf{1}, \mathbf{0}, \mathbf{1}, \mathbf{0}, \quad 0, 0, 0, 0] \\ \phi(x, \text{OTHER}) &= [0, 0, 0, 0, \quad 0, 0, 0, 0, \quad \mathbf{1}, \mathbf{0}, \mathbf{1}, \mathbf{0}]\end{aligned}$$

Linear Scoring & Prediction

In a linear model, each feature dimension in $\phi(x, y)$ is assigned a learned real-valued weight in weight vector $w \in \mathbb{R}^{|L| \cdot d}$.

- Linear Score:
$$\text{score}(x, y; w) = w \cdot \phi(x, y) = \phi(x, y)^T w$$
- Deterministic Decision Rule: To classify an input x, predict the label y^* that maximizes the linear compatibility score:
$$y^* = \arg\max_{y \in L} w \cdot \phi(x, y)$$

Numerical Scoring Example

Let the learned weights partitioned by category blocks be:

$$w = [\underbrace{1, 1, -1, -2}_{\text{SPORTS}}, \quad \underbrace{1, -1, 1, -2}_{\text{POLITICS}}, \quad \underbrace{-2, -1, -1, 1}_{\text{OTHER}}]$$

Evaluating the candidate scores:

- $\phi(x, \text{SPORTS})^T w = (1 \cdot 1) + (0 \cdot 1) + (1 \cdot -1) + (0 \cdot -2) = 1 - 1 = 0$
- $\phi(x, \text{POLITICS})^T w = (1 \cdot 1) + (0 \cdot -1) + (1 \cdot 1) + (0 \cdot -2) = 1 + 1 = 2$

Because $\phi(x, \text{POLITICS})^T w > \phi(x, \text{SPORTS})^T w$, the model predicts $\text{Prediction}(x) = \text{POLITICS}$.

Log-Linear Models (Multinomial Logistic Regression / Softmax)

To convert arbitrary real-valued scores $\text{score}(x, y; w) \in (-\infty, \infty)$ into a normalized conditional probability distribution $P(y \mid x; w)$, we exponentiate and normalize (the Softmax function):

$$P(y \mid x; w) = \frac{\exp(w \cdot \phi(x, y))}{\sum_{y' \in L} \exp(w \cdot \phi(x, y'))}$$

- Prediction: Compute $P(y \mid x; w)$ for each candidate label $y \in L$ and select the maximum:
$$y^* = \arg\max_{y \in L} P(y \mid x; w) = \arg\max_{y \in L} (w \cdot \phi(x, y))$$
- (Since exponentiation and division by the constant normalizer do not change the ranking of labels).

Learning: Maximum Conditional Likelihood

Given n independent labeled training pairs $\{(x_1, y_1), \ldots, (x_n, y_n)\}$, parameters w are estimated by maximizing the conditional log-likelihood of the training data:

$$L(w) = \log \prod_{i=1}^{n} P(y_i \mid x_i; w) = \sum_{i=1}^{n} \log P(y_i \mid x_i; w)$$

$$w^* = \arg\max_{w} L(w)$$

- Convexity / Concavity: The objective $L(w)$ is strictly concave with respect to w (equivalently, negative log-likelihood is convex).
- Global Optimum: Because the objective is concave, it has no local optimastandard convex optimization algorithms (such as Gradient Ascent, L-BFGS / quasi-Newton methods, or SGD) are guaranteed to converge to the global maximum.

The Gradient and Its Interpretation

Taking the partial derivative of $L(w)$ with respect to weight parameter w_j yields an elegant, intuitive formulation:

$$\frac{\partial L(w)}{\partial w_j} = \sum_{i=1}^{n} \left(\phi_j(x_i, y_i) - \sum_{y' \in L} P(y' \mid x_i; w) \phi_j(x_i, y') \right)$$

$$\frac{\partial L(w)}{\partial w_j} = \underbrace{\sum_{i=1}^{n} \phi_j(x_i, y_i)}_{\text{Total empirical count of feature } j \text{ in correct labels}} - \underbrace{\sum_{i=1}^{n} \mathbb{E}_{y' \sim P(y' \mid x_i; w)}[\phi_j(x_i, y')]}_{\text{Expected count of feature } j \text{ under the current model distribution}}$$

- Gradient at Convergence: At the optimal parameter set w^*, the gradient equals zero ($\nabla_w L(w^*) = 0$): the expected feature counts predicted by the model match the empirical feature counts observed in the training data exactly.

Step-by-Step Derivation of the Gradient

Expanding the log-likelihood for the training set:

$$L(w) = \sum_{i=1}^{n} \log \left(\frac{\exp(w \cdot \phi(x_i, y_i))}{\sum_{y' \in L} \exp(w \cdot \phi(x_i, y'))} \right)$$

Using the log quotient identity $\log(a/b) = \log a - \log b$:

$$L(w) = \sum_{i=1}^{n} \left(w \cdot \phi(x_i, y_i) - \log \sum_{y' \in L} \exp(w \cdot \phi(x_i, y')) \right)$$

Differentiating term-by-term with respect to coordinate w_j:

1. Derivative of the first term:
$$\frac{\partial}{\partial w_j}[w \cdot \phi(x_i, y_i)] = \phi_j(x_i, y_i)$$

2. Derivative of the normalizer (chain rule):
$$\begin{aligned} \frac{\partial}{\partial w_j} \left[\log \sum_{y' \in L} \exp(w \cdot \phi(x_i, y')) \right] &= \frac{1}{\sum_{y' \in L} \exp(w \cdot \phi(x_i, y'))} \cdot \frac{\partial}{\partial w_j} \left(\sum_{y'' \in L} \exp(w \cdot \phi(x_i, y'')) \right) \\ &= \frac{1}{\sum_{y' \in L} \exp(w \cdot \phi(x_i, y'))} \cdot \sum_{y'' \in L} \exp(w \cdot \phi(x_i, y'')) \cdot \phi_j(x_i, y'') \\ &= \sum_{y'' \in L} \phi_j(x_i, y'') \cdot \frac{\exp(w \cdot \phi(x_i, y''))}{\sum_{y' \in L} \exp(w \cdot \phi(x_i, y'))} \\ &= \sum_{y'' \in L} P(y'' \mid x_i; w) \cdot \phi_j(x_i, y'') \end{aligned}$$

3. Combining across all n samples:
$$\frac{\partial L(w)}{\partial w_j} = \sum_{i=1}^{n} \left(\phi_j(x_i, y_i) - \sum_{y'' \in L} P(y'' \mid x_i; w) \phi_j(x_i, y'') \right)$$

Regularization in Log-Linear Models

Unlike generative models, log-linear models do not collapse when faced with zero countsunseen feature combinations simply do not contribute positively to the linear score rather than zeroing out the entire calculation.

However, they suffer from a distinct vulnerability: weight explosion and overfitting.

- If a rare feature appears only once in the training set and happens to co-occur with the class SPORTS, pure maximum likelihood estimation incentivizes the optimizer to push the corresponding weight toward positive infinity ($w_j \to +\infty$) to maximize the probability assigned to that single training example.

- To prevent parameters from growing unbounded and overfitting sparse noise, we add a regularization penaltymost commonly L_2 regularization (Ridge penalty / Gaussian prior):

$$L(w) = \sum_{i=1}^{n} \log P(y_i \mid x_i; w) - \frac{\lambda}{2}\|w\|^2$$

- The hyperparameter $\lambda > 0$ controls the regularization strength, penalizing large weights and balancing empirical fit against model complexity.

Regularized Objective & Modified Gradient

Expanding the regularized conditional log-likelihood:

$$L(w) = \sum_{i=1}^{n} \left(w \cdot \phi(x_i, y_i) - \log \sum_{y \in L} \exp(w \cdot \phi(x_i, y)) \right) - \frac{\lambda}{2}\|w\|^2$$

Taking the partial derivative with respect to an individual coordinate w_j yields the modified gradient:

$$\frac{\partial L(w)}{\partial w_j} = \underbrace{\sum_{i=1}^{n} \phi_j(x_i, y_i)}_{\text{Empirical count in correct candidates}} - \underbrace{\sum_{i=1}^{n} \sum_{y \in L} P(y \mid x_i; w)\phi_j(x_i, y)}_{\text{Expected count under current model}} - \underbrace{\lambda w_j}_{\text{Weight decay penalty}}$$

- Empirical Count: The actual number of times feature j was observed with the true labels across the training corpus.
- Model Expectation: The expected number of times feature j would be activated under the model's current predicted label probabilities.
- $-\lambda w_j$ ("Big weights are bad"): A linear decay force pulling large weights back toward zero at every step, directly discouraging extreme parameter values.

Stochastic Gradient Ascent (SGD) for Log-Linear Models

In practice, evaluating the gradient over the entire corpus of N documents for batch gradient ascent is computationally expensive. We instead apply Stochastic Gradient Ascent (SGD), updating weights after observing one example at a time:

SGD Algorithm

1. `Initialize` $w \leftarrow \vec{0}$
2. `for` $r = 1 \dots N_{\text{iterations}}$:
3. `for` $i = 1 \dots N$:
4. $w \leftarrow w + \eta * \left(\phi(x_i, y_i) - \mathbb{E}_{\hat{y} \sim P(\cdot \mid x_i; w)}[\phi(x_i, \hat{y})]\right)$
5. `return` w

(where η is the learning rate, and the optional L_2 regularization term updates as $w \leftarrow (1 - \eta\lambda)w + \dots$).

Comparison: Log-Linear SGD vs. The Structured Perceptron

The inner update step reveals the fundamental relationship between log-linear models (maximum entropy) and the classical Perceptron algorithm:

- Log-Linear SGD Update:

$$w \leftarrow w + \eta \left(\phi(x_i, y_i) - \mathbb{E}_{\hat{y} \sim P(\hat{y} \mid x_i; w)}[\phi(x_i, \hat{y})]\right)$$

- The model subtracts a soft, probabilistic expectation over all candidate classes weighted by their probabilities.
- Perceptron Update:

$$w \leftarrow w + \eta \left(\phi(x_i, y_i) - \phi(x_i, \hat{y}_{\text{best}})\right) \quad \text{where } \hat{y}_{\text{best}} = \arg\max_{y}(w \cdot \phi(x_i, y))$$

- Instead of computing an expectation across the entire label space, the Perceptron takes a hard arg max, updating weights solely based on the single top-scoring predicted candidate.

Concrete Applications of Feature-Based Classification

1. Named Entity Recognition (NER)

- The Task: Identify all entity name mentions in unstructured text and classify them into predefined semantic categories (e.g., Location, Organization, Person, Time, Money).
- Example Annotation (Jurafsky & Martin): Citing high fuel prices, [ORG United Airlines] said [TIME Friday] it has increased fares by [MONEY $6] per round trip on flights to some cities also served by lower-cost carriers. [ORG American Airlines], a unit of [ORG AMR Corp.], immediately matched the move, spokesman [PER Tim Wagner] said. [ORG United], a unit of [ORG UAL Corp.], said the increase took effect [TIME Thursday] and applies to most routes where it competes against discount carriers, such as [LOC Chicago] to [LOC Dallas] and [LOC Denver] to [LOC San Francisco].

Regularization in Action: NER Disambiguation

When training a feature-rich classifier (like a log-linear model) on local context windows, multiple overlapping features compete for credit:

- Local Context Example: Classifying the word "Grace" in the context fragment "at Grace Road":
  - Tokens: Previous = "at", Current = "Grace", Next = "Road"
  - Part-of-Speech Tags: Previous = IN (preposition), Current = NNP (proper noun), Next = NNP
  - Word Shape Signatures: Previous = x (lowercase), Current = Xx (capitalized), Next = Xx
- Feature Weights and Total Scores:

Feature Type	**Feature Value**	**Weight(PER)**	**Weight(LOC)**
Previous word	"at"	-0.73	$+0.94$
Current word	"Grace"	$+0.03$	0.00
Beginning bigram (prefix)	"Gr"	$+0.45$	-0.04
Current POS tag	NNP	$+0.47$	$+0.45$
Previous & current tags	IN NNP	-0.10	$+0.14$
Current shape signature	Xx	$+0.80$	$+0.46$
Context signature	x-Xx-Xx	-0.69	$+0.37$
Previous state + signature	O-x-Xx	-0.20	$+0.82$
...	...	...	...
Total Linear Score		$\mathbf{-0.58}$	$\mathbf{+2.68}$

- Why Regularization Matters Here: While the exact token "Grace" is specific, it appears infrequently in training, so L_2 regularization penalizes assigning it an excessively large weight.
  - Broader, more frequent contextual features (such as the prefix "at", preceding preposition tags, and subsequent noun patterns) accumulate stronger, reliable weights.
  - Consequently, the total score strongly favors LOC (2.68) over PER (-0.58), correctly identifying "Grace Road" as a location rather than a person's name.

2. Sentiment Analysis

Sentiment analysis frames text classification around user appraisal, opinion, and affect:

- Example 1: Document/Sentence-Level Review Polarity
  - Negative ($\ominus$): "Unbelievably disappointing"
  - Positive ($\oplus$): "Full of zany characters and richly applied satire, and some great plot twists"
  - Positive ($\oplus$): "This is the greatest screwball comedy ever filmed"
  - Negative ($\ominus$): "It was pathetic. The worst part about it was the boxing scenes."
- Example 2: Aspect-Based Product Reviews Instead of a single scalar label, reviews are classified along fine-grained aspects (e.g., evaluating an HP printer):
  - Ease of use: "This was very easy to setup to four computers."
  - Value: "Appreciate good quality at a fair price."
  - Customer service: "I DO like honest tech support people."
  - Print mode / color quality: "Full color prints came out with great quality."
- Macro Impact: Market Predictability

  - Bollen et al. (2011) showed that aggregate sentiment measured from large-scale social text (e.g., the Twitter mood extraction system CALM) exhibited predictive correlation with the Dow Jones Industrial Average (DJIA) closing values up to 3 days later.
  - Such discriminative sentiment classifiers serve as real-time feature feeds for algorithmic hedge fund trading strategies.

Psycholinguistic Grounding: Scherers Typology of Affective States

In affective computing and computational linguistics, Klaus Scherer distinguishes five distinct categories of affective states:

1. Emotion: Brief, organically synchronized evaluation of a major event (e.g., angry, sad, joyful, fearful, ashamed, proud, elated).
2. Mood: Diffuse, non-caused, low-intensity, long-duration shift in subjective feeling (e.g., cheerful, gloomy, irritable, listless, depressed, buoyant).
3. Interpersonal Stances: Affective stance adopted toward another specific person during interaction (e.g., friendly, flirtatious, distant, cold, warm, supportive, contemptuous).
4. Attitudes (The Core Target of Sentiment Analysis): Enduring, affectively colored beliefs and dispositions toward specific objects, persons, or propositions (e.g., liking, loving, hating, valuing, desiring).
5. Personality Traits: Stable, long-term personality dispositions and behavioral tendencies (e.g., nervous, anxious, reckless, morose, hostile, jealous).

Standard NLP sentiment analysis tasks primarily operationalize attitudespredicting an agent's positive or negative appraisal toward an entity, product, or topic.

Sentiment Analysis: Formal Definition & Components

Drawing on Scherers typology, sentiment analysis is computationally framed as the detection and extraction of attitudesdefined as enduring, affectively colored beliefs or dispositions towards objects, persons, or propositions:

1. Holder (Source): The person, entity, or group expressing the attitude.
2. Target (Aspect): The entity, event, or specific attribute receiving the attitude (e.g., the screen quality of a phone, the plot of a film).
3. Type of Attitude:
   - Qualitative / Affective Categories: A set of discrete dispositions (e.g., like, love, hate, value, desire).
   - Weighted Polarity (Standard Formulation): Categorized along a spectrum of positive, negative, or neutral, often paired with an intensity/strength score.
4. Text Span: The textual scope expressing the evaluation, ranging from a phrase or clause to an entire sentence or document.

Task Spectrum: Coarse to Fine-Grained

- Binary Polarity (Simplest): Deciding whether an input text conveys an overall positive or negative attitude ($\{+1, -1\}$).
- Ordinal Rating / Regression: Mapping text to a discrete rating scale (e.g., 1 to 5 stars on an e-commerce review).
- Aspect-Based & Target Extraction (Advanced): Jointly extracting the target entity, holder, and fine-grained sentiment toward sub-components.
- Sub-Sentential & Compositional Sentiment: Assigning sentiment values not only to full sentences, but recursively to clauses and phrases:
  - Example: "[The pizza was overall bad], but the toppings were fun!" $\rightarrow$ The first clause is negative, the second positive, yet the discourse contrast yields an overall negative assessment of the primary subject.

Benchmark Case: Movie Review Classification

A foundational milestone in statistical NLP was applying supervised machine learning to IMDB movie review sentiment:

- Seminal Literature: Pang, Lee, and Vaithyanathan (2002): Thumbs up? Sentiment Classification using Machine Learning Techniques (EMNLP).

  - Pang and Lee (2004): A Sentimental Education: Sentiment Analysis Using Subjectivity Summarization Based on Minimum Cuts (ACL).
- Review Challenges: Positive reviews frequently spend long narrative paragraphs summarizing bleak plotlines before praising the cinematography: "...when Han Solo goes light speed, the stars change to bright lines... cool." ($\oplus$)
  - Negative reviews often open with positive-sounding words or high expectations before delivering the verdict: "...the most aggravating kind of movie: the kind that shows so much potential then becomes unbelievably disappointing... since he's a great director..." ($\ominus$)

Building a Simple Sentiment Classifier

- Core Architectures: Multinomial Naïve Bayes or regularized Log-Linear (Multinomial Logistic Regression) models.
- Feature Design: Word identity (unigrams, bigrams).
  - Domain-specific markup (e.g., social media hashtags, HTML formatting, capitalization patterns, exclamation marks).
- Binary Indicators vs. Raw Frequencies: In sentiment classification, encoding features as binary presence/absence flags ($x_j \in \{0, 1\}$) almost always outperforms raw word counts ($c(w)$).
  - Non-Linear Utility: Saying the word "terrible" once strongly signals negative sentiment; saying it five times conveys redundant semantic information rather than five times the negative weight. Binary indicators avoid skewing linear models with word-frequency inflation.

Error Analysis: Why Sentiment Classification is Hard

1. Sarcasm and Irony

Surface lexical choices often directly contradict the author's underlying intent:

- Perfume Review: "If you are reading this because it is your darling fragrance, please wear it at home exclusively, and tape the windows shut."
- (Uses positive/intimate phrasing like "darling fragrance" to deliver a biting negative critique).
- Automobile Steering Wheel Work Surface Review: "You wouldn't believe how much more interesting my commute is now that I have something to do other than just stare out the window! I'm using it right now to post this review and I never..."
- (Enthusiastic surface tone masking obvious satirical absurdity).

2. Thwarted Expectations and Discourse Ordering

Reviews often dedicate significant token volume to praising attributes that were expected to succeed, only to reverse the entire conclusion in the final clause:

- Review Text: "This film should be brilliant. It sounds like a great plot, the actors are first grade, and the supporting cast is good as well, and Stallone is attempting to deliver a good performance. However, it can't hold up."
- The Failure Mode: Bag-of-words classifiers accumulate strong positive weights from "brilliant", "great plot", "first grade", and "good performance", overwhelming the final negation and misclassifying the document as positive.

Handling Negation

Negation radically flips sentiment polarity, requiring explicit modeling strategies.

The Heuristic Approach: Dashed Negation (NOT_ Prefix)

A classical baseline proposed by Das and Chen (2001) appends a prefix NOT_ to every word occurring between a negation marker and the nearest following punctuation mark:

```
Original:    didn't like this movie , but I
Transformed: didn't NOT_like NOT_this NOT_movie , but I
```

This ensures that the feature `NOT_like` receives an independent negative weight rather than activating the positive weight of `like`.

Limitations of Heuristic Negation

This regex-style heuristic remains fundamentally brittle:

1. Implicit / Lexical Negation: Polarities are frequently inverted without overt negative particles (e.g., no, not):
   - "He failed to convey the importance of his message."
2. Complex Logical & Modal Structures: Deep syntactic contexts alter standard compositional polarity:
   - "I don't think anyone could have done a better job." (Explicit negation syntactically attached to "think", yet conveying glowing praise).
3. Negation Scope Boundaries: Punctuation delimiters fail to capture true syntactic clause boundaries:
   - "I didn't like the exposition in this otherwise excellent film." (Heuristic scoping would blindly negate "otherwise" and "excellent", reversing praise into criticism).
4. Modern Solution: Resolving scope ambiguity requires moving beyond bag-of-words heuristics to structural models that encode surrounding local contextsuch as dependency parse trees, convolutional representations, and modern bidirectional transformer encoders.